

iptables 使用方式整理

📅 Jul 21, 2019 | 📁 计算机科学 | 💎 iptables

`iptables` 是 Linux 系统上的 IP 信息包过滤系统。如果 Linux 系统连接到因特网或 LAN、服务器或连接 LAN 和因特网的代理服务器，则该系统有利于在 Linux 系统上更好地控制 IP 信息包过滤，其可以说是 Linux 的网络防火墙。广义上的 iptables 实际上是由 **netfilter** 和 **iptables** 两个组件组成。而狭义上的 `iptables` 是指一个命令行工具，用于配置管理信息包的过滤规则。真正起到信息包过滤作用的是 netfilter 组件。

netfilter/iptables

`netfilter` 是内核的一个子系统，其工作在内核空间，核心是一个报文过滤架构，它包含了一组分布在报文处理各个阶段的钩子函数，报文经过网络协议栈时进入 netfilter 处理架构，会调用其他模块在各个阶段注册的钩子函数，并返回处理结果，netfilter 根据返回结果进行不同的处理。

报文处理的五个不同阶段：

- (1) pre-routing: 经过 ip 合法性检查的报文
- (2) local-input: 经过选路后，目的地址指向本机的报文
- (3) forward: 经过选路需要转发出去的报文
- (4) local-output: 从本协议栈发送出去的报文
- (5) post-routing: 经过选路后需要转发出去的报文，最终送给这个钩子函数处理

`iptables` 工作在用户空间，用于设置、维护和检查信息包的过滤规则，其是一个与 netfilter 子系统交互的工具，它使插入、修改和除去信息包过滤表中的规则变得容易。由于在使用过程中通常只接触 iptables 这个命令工具，所以通常所说的 iptables 即指 netfilter 子系统与 iptables 工具组成的信息包过滤系统。

表/链/规则

使用 iptables 需要先理解表 (table)、链 (chain)、规则 (rule) 这三个概念。`netfilter/iptables` 系统可以理解成是 **表** 的容器，这也是它被称为 iptables 的原因，而表则是 **链** 的容器，即所有的链都属于其对应的表，链又是 **规则** 的容器。

表(tables)

`iptables` 大致有 raw, filter, nat, mangle, security 等五类表，常用的表有 filter、nat、mangle 三个表。

- **raw 表：** 用于配置数据包，raw 中的数据包不会被系统跟踪。其优先级最高，设置 raw 时一般是为了不再让 iptables 做数据包的链接跟踪处理，提高性能
- **filter 表：** 为 iptables 默认的表，在操作时如果没有指定使用哪个表，iptables 默认使用 filter 表来执行所有的命令。filter 表根据预定义的一组规则过滤符合条件的数据包。在 filter 表中只允许对数据包进行接收、丢弃的操作，而无法对数据包进行更改
- **nat 表：** 即 Network Address Translation，主要是用于网络地址转换（例如：端口转发），该表可以实现一对一、一对多、多对多等 NAT 工作
- **mangle 表：** 主要用于对指定包的传输特性进行修改。某些特殊应用可能需要改写数据包的一些传输特性，例如更改数据包的 TTL 和 TOS 等
- **security 表：** 用于强制访问控制网络规则（例如：SELinux）

在大多数使用情况下都不会用到 raw 和 security 表，而 filter 和 nat 表则是会常用到的，mangle 表次之。

链(chains)

链(chains)是数据包传输的路径，对应着前面提到的报文处理的五个阶段，也相当于五个不同的关卡：

- **INPUT：** 处理入站数据包，当接收到访问本机地址的数据包(入站)时，应用此链中的规则
- **OUTPUT：** 处理出站数据包，当本机向外发送数据包(出站)时，应用此链中的规则
- **FORWARD：** 处理转发数据包，当接收到需要通过本机发送给其他地址的数据包(转发)时，应用此链中的规则
- **PREROUTING：** 在对数据包作路由选择之前，应用此链中的规则
- **POSTROUTING：** 在对数据包作路由选择之后，应用此链中的规则

链是规则的容器，一条链中可能包含着众多的规则，当一个数据包到达一个链时，iptables 就会从链中第一条规则开始匹配，如果满足该规则的条件，系统就会根据该条规则所定义的方法处理该数据包，否则将继续匹配下一条规则，如果该数据包不符合链中任一条规则，iptables 就会根据该链预先定义的默认策略来处理数据包。

INPUT, OUTPUT 链更多的应用在本机的网络控制中，即主要针对本机进出数据的安全控制。而 FORWARD, PREROUTING, POSTROUTING 链更多地应用在对我的网络控制中，特别是机器作为网关使用时的情况。

表是链的容器，不同的表中包含着不同的链：

Filter 表 是 iptables 的默认表，因此如果没有指定表，那么默认操作的是 filter 表，其包含以下三种内建链：

- INPUT 链 – 处理来自外部的数据
- OUTPUT 链 – 处理向外发送的数据
- FORWARD 链 – 将数据转发到本机的其他网卡设备上

NAT 表 包含以下三种内建链：

- PREROUTING 链 – 处理刚到达本机并在路由转发前的数据包。它会转换数据包中的目标 IP 地址 (destination ip address)，通常用于 DNAT(destination NAT)

- POSTROUTING 链 – 处理即将离开本机的数据包。它会转换数据包中的源 IP 地址（source ip address），通常用于 SNAT（source NAT）
- OUTPUT 链 – 处理本机产生的数据包

Mangle 表 指定如何处理数据包。它能改变 TCP 头中的 QoS 位。Mangle 表包含五种内建链：PREROUTING, OUTPUT, FORWARD, INPUT, POSTROUTING.

Raw 表 包含两个内建链：PREROUTING, OUTPUT.

规则(rules)

规则(rules)是一些预定义的数据包过滤条件。规则存储在内核空间的信息包过滤表中，数据包每经过一个链(关卡)时，系统会根据链中规则指定的匹配条件来尝试匹配，一旦匹配成功，则由规则后面指定的处理动作进行处理。

规则分别指定了源地址、目的地址、传输协议（如TCP、UDP、ICMP）和服务类型（如HTTP、FTP和SMTP）等。当数据包与规则匹配时，iptables就根据规则所定义的方法来处理这些数据包，如放行（accept）、拒绝（reject）和丢弃（drop）等。配置防火墙的主要工作就是添加、修改和删除这些规则。

规则由 **匹配条件** 和 **处理动作** 组成。**匹配条件** 又分为 **基本匹配条件** 与 **扩展匹配条件**。基本匹配条件如：源地址 Source IP，目标地址 Destination IP；扩展匹配条件通常以模块的形式存在，这些模块可以按需安装，源端口 Source Port，目标端口 Destination Port。

处理动作(target) 也分为基本动作和扩展动作。以下列举一些常用的动作：

- **ACCEPT**：允许数据包通过
- **DROP**：直接丢弃数据包，不给任何回应信息，这时候客户端会感觉自己的请求泥牛入海了，过了超时时间才会有反应
- **QUEUE**：将数据包移交到用户空间
- **RETURN**：停止执行当前链中的后续规则，并返回到调用链(The Calling Chain)中
- **REJECT**：拒绝数据包通过，必要时会给数据发送端一个响应的信息，客户端刚请求就会收到拒绝的信息
- **DNAT**：目标地址转换
- **SNAT**：源地址转换，解决内网用户用同一个公网地址上网的问题
- **MASQUERADE**：是 SNAT 的一种特殊形式，适用于动态的、临时会变的 ip 上
- **REDIRECT**：在本机做端口映射
- **LOG**：记录日志信息，除记录外不对数据包做任何其他操作，仍然匹配下一条规则

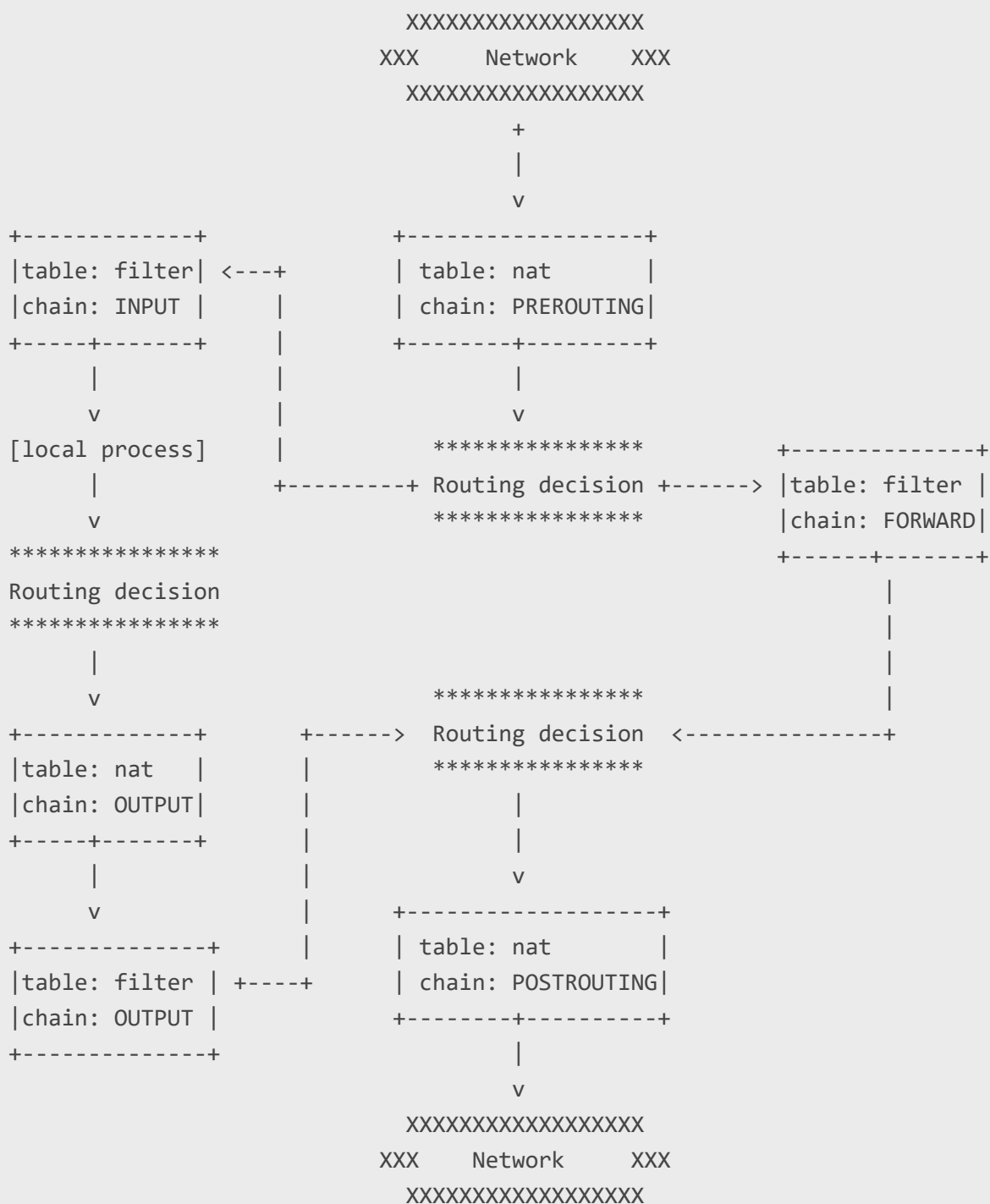
由此，已知 tables 由 chains 组成，而 chains 又由 rules 组成。常用的表有 filter、nat、mangle 三种，链有五种，对应报文处理的五个阶段。对规则理解的关键则需记住以下三点：

- 1、一条规则包括一个条件和一个动作(target)
- 2、如果满足条件，就执行处理动作
- 3、如果不满足条件，就继续匹配下一条规则

几个常用表单的优先级顺序是 mangle --> nat --> filter.

数据包流向

以下简略图描述了网络数据包经过 netfilter/iptables 的过程:



由图可知，当一个数据包进入计算机的网络接口时，数据首先进入 POSTROUTING 链，然后内核根据路由表决定数据包的目标。若数据包的目的地址是本机，则将数据包送往 INPUT 链进行规则匹配，当数据包进入 INPUT 链后，系统的任何进程都可以收到它，本机上运行的程序可以发送该数据包，这些数据包会经过 OUTPUT 链，再从 POSTROUTING 链发出；若数据包的目的地址不是本机，则检查内核是否允许转发，若允许，则将数据包送 FORWARD 链进行规则匹配，若不允许，则丢弃该数据包。若是主机本地进程产生并准备发出的包，则数据包被送往 OUTPUT 链进行规则匹配。

- 本机发出的包：本机进程 -> OUTPUT 链 -> 路由选择 -> POSTROUTING 链 -> 出口网卡
- 本机收到的包：入口网卡 -> PREROUTING 链 -> 路由选择 -> 此时有两种可能的情况：
 - 目的地址为本机：INPUT 链 -> 本机进程

- 目的地址不为本机：FORWARD 链 -> POSTROUTING 链 -> 网卡出口（内核允许网卡转发的情况下）

使用简介

`iptables` 工具的操作命令是比较复杂的，但大致的格式如下所示：

```
iptables [-t 表]
          命令选项
          [链]
          [匹配选项]
          [操作选项]
```

命令选项：

选项名	功能及特点
-A --append	在指定链的末尾添加一条新的规则
-D --delete	删除指定链中的某一条规则，按规则序号或内容确定要删除的规则
-I --insert	在指定链中插入一条新的规则，默认在链的开头插入
-R --replace	修改、替换指定链中的一条规则，按规则序号或内容确定
-F --flush	清空指定链中的所有规则，默认清空表中所有链的内容
-N --new	新建一条用户自己定义的规则链
-X --delete-chain	删除指定表中用户自定义的规则链
-P --policy	设置指定链的默认策略
-F, --flush	清空指定链上面的所有规则，如果没有指定链，清空表上所有链的所有规则
-Z, --zero	把指定链或表中的所有链上的所有计数器清零
-L --list	列出指定链中的所有的规则进行查看，默认列出表中所有链的内容
-S --list-rules	以原始格式列出链中所有规则
-v --verbose	查看规则列表时显示详细的信息
-n --numeric	用数字形式显示输出结果，如显示主机的 IP 地址而不是主机名
--line-number	查看规则列表时，同时显示规则在链中的顺序号

匹配选项：

选项名	功能及特点
-i --in-interface	匹配输入接口，如 eth0, eth1

选项名	功能及特点
-o --out-interface	匹配输出接口
-p --proto	匹配协议类型，如 TCP、UDP 和 ICMP等
-s --source	匹配的源地址
--sport	匹配的源端口号
-d --destination	匹配的目的地地址
--dport	匹配的目的地端口号
-m --match	匹配规则所使用的过滤模块

部分过滤模块使用简介：

- **-m limit --limit**

```
iptables -A INPUT -m limit --limit 3/hour
```

匹配某段时间内封包的平均流量，以上示例匹配：每小时平均流量是否超过一次 3 个封包。除了每小时平均次外，也可以每秒钟、每分钟或每天平均一次，默认值为每小时平均一次，参数如后： /second、 /minute、 /day。除了进行封包数量的匹配外，设定这个参数也会在条件达成时，暂停封包的匹配动作，以避免因骇客使用洪水攻击法，导致服务被阻断。

- **-m limit --limit-burst**

```
iptables -A INPUT -m limit --limit-burst 5
```

匹配瞬间大量封包的数量，上例匹配一次同时涌入的封包是否超过 5 个（这是默认值），超过此上限的封包将被直接丢弃。使用效果同上。

- **-m mac --mac-source**

```
iptables -A INPUT -m mac --mac-source 00:00:00:00:00:01
```

匹配封包来源网络接口的硬件地址，这个参数不能用在 OUTPUT 和 POSTROUTING 规则链上，因为封包要送到网卡后，才能由网卡驱动程序透过 ARP 通讯协议查出目的地的 MAC 地址，所以 iptables 在进行封包匹配时，并不知道封包会送到哪个网络接口去。

- **-m mac --mark**

```
iptables -t mangle -A INPUT -m mark --mark 1
```

匹配封包是否被表示某个号码，当封包被匹配成功时，可以透过 MARK 处理动作，将该封包标示一个号码，号码最大不可以超过 4294967296。

- **-m owner --uid-owner**

```
iptables -A OUTPUT -m owner --uid-owner 500
```

匹配来自本机的封包，是否为某特定使用者所产生的，这样可以避免服务器使用 root 或其它身分将敏感数据传送出，可以降低系统被黑的损失。可惜这个功能无法匹配来自其它主机的封包。

- **-m owner --gid-owner**

```
iptables -A OUTPUT -m owner --gid-owner 0
```

匹配来自本机的封包，是否为某特定使用者群组所产生的，使用时机同上。

- **-m owner --pid-owner**

```
iptables -A OUTPUT -m owner --pid-owner 78
```

匹配来自本机的封包，是否为某特定进程所产生的，使用时机同上。

- **-m owner --sid-owner**

```
iptables -A OUTPUT -m owner --sid-owner 100
```

匹配来自本机的封包，是否为某特定 连接（Session ID）的响应封包，使用时机同上。

- **-m state --state**

```
iptables -A INPUT -m state --state RELATED,ESTABLISHED
```

匹配连接状态，连接状态共有四种：INVALID、ESTABLISHED、NEW 和 RELATED。**INVALID:** 表示该封包的连接编号（Session ID）无法辨识或编号不正确；**ESTABLISHED:** 表示该封包属于某个已经建立的连接；**NEW:** 表示该封包想要起始一个连接（重设连接或将连接重导向）；**RELATED** 表示该封包是属于某个已经建立的连接，所建立的新连接。

操作选项 一般为 `-j 处理动作` 的形式，处理动作包括 ACCEPT、DROP、RETURN、REJECT、DNAT、SNAT 等。不同的处理动作可能还有额外的选项参数，如指定 DNAT、SNAT 动作则还需指定 `--to` 参数用以说明要替换的地址，指定 REDIRECT 动作则需指定 `--to-ports` 参数用于说明要跳转的端口。

使用 `iptables -j 动作名 --help` 可以查看处理动作帮助。

常用命令

查看帮助:

```
iptables --help          # 查看 iptables 的帮助
iptables -m 模块名 --help # 查看指定模块的可用参数
iptables -j 动作名 --help # 查看指定动作的可用参数
```

查看规则:

```
iptables -nvL
iptables -t nat -nvL

# 显示规则序号
iptables -nvL INPUT --line-numbers
iptables -t nat -nvL --line-numbers
iptables -t nat -nvL PREROUTING --line-numbers

# 查看规则的原始格式
iptables -t filter -S
iptables -t nat -S
iptables -t mangle -S
iptables -t raw -S
```

清除所有规则:

```
iptables -F # 清空表中所有的规则
iptables -X # 删除表中用户自定义的链
iptables -Z # 清空计数

iptables -t nat -F
iptables -t nat -X
iptables -t mangle -F
iptables -t mangle -X
iptables -t raw -F
iptables -t raw -X
iptables -t security -F
iptables -t security -X
```

设置默认规则:

```
iptables -P INPUT DROP # 配置默认丢弃访问的数据表
iptables -P FORWARD DROP # 配置默认禁止转发
iptables -P OUTPUT ACCEPT # 配置默认允许向外的请求
```

增加、删除、修改规则:


```
# 增加一条规则到最后
iptables -A INPUT -i eth0 -p tcp --dport 80 -m state --state NEW,ESTABLISHED -j ACCEPT

# 注：以下几条操作都需要使用规则的序号，需要使用 -L --line-numbers 参数先查看规则的顺序号

# 添加一条规则到指定位置
iptables -I INPUT 2 -i eth0 -p tcp --dport 80 -m state --state NEW,ESTABLISHED -j ACCEPT

# 删除一条规则
iptables -D INPUT 2

# 修改一条规则
iptables -R INPUT 3 -i eth0 -p tcp --dport 80 -m state --state NEW,ESTABLISHED -j ACCEPT
```

开放指定端口：

```
# 允许本地回环接口(即运行本机访问本机)
iptables -A INPUT -i lo -j ACCEPT

# 允许已建立的或相关连接的通行
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# 允许所有本机向外的访问
iptables -A OUTPUT -j ACCEPT

# 允许 22,80,443 端口的访问
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -p tcp --dports 80,443 -j ACCEPT

# 如果有其他端口的需要开放，则同上
iptables -A INPUT -p tcp --dport 8000:8010 -j ACCEPT

# 允许 ping
iptables -A INPUT -p icmp -m icmp --icmp-type 8 -j ACCEPT

# 禁止其他未允许的规则访问
iptables -A INPUT -j REJECT
iptables -A FORWARD -j REJECT

# 注：以上操作需要安装顺序，确保规则顺序正确
```

目的地址转换，首先需要在开启中开启转发功能（源地址转换也需要开启）：

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

```
# 把从 eth0 进来要访问 TCP/80 的数据包的地址转换到 192.168.1.18
iptables -t nat -A PREROUTING -p tcp -i eth0 --dport 80 -j DNAT --to 192.168.1.18

# 把从 123.57.172.149 进来要访问 TCP/80 的数据包的地址转换到 192.168.1.118:8000
```

```
iptables -t nat -A PREROUTING -p tcp -d 123.57.172.149 --dport 80 -j DNAT --to 192.168.1.118:8000
```

源地址转换：

```
# 最典型的应用是让内网机器可以访问外网：
# 将内网 192.168.0.0/24 的源地址修改为 1.1.1.1 (可以访问互联网的机器的 IP)
iptables -t nat -A POSTROUTING -s 192.168.0.0/24 -j SNAT --to 1.1.1.1

# 将内网机器的源地址修改为一个 IP 地址池
iptables -t nat -A POSTROUTING -s 192.168.0.0/24 -j SNAT --to 1.1.1.1-1.1.1.10
```

持久化规则：

```
# 保存当前规则
iptables-save > iptables.20190721

# 恢复备份规则
iptables-restore < iptables.20190721
```

参考资料

- [Wiki Iptables](#)
- [Arch wiki Linux Iptables](#)
- [服务器 VPS 安全防护](#)
- <https://wangchujang.com/linux-command/c/iptables.html>
- <https://www.cnblogs.com/Dicky-Zhang/p/5904429.html>
- <https://www.zfl9.com/iptables.html>

\$ 您的支持是作者写作最大的动力!

本文使用 [知识共享署名-非商业性使用-禁止演绎 4.0 国际许可协议](#) 进行许可，转载请注明出处

◀ Python 模块简介 -- inspect

DNS 域名解析系统概述 ▶